



KOCAELİ ÜNİVERSİTESİ

Mühendislik Fakültesi – Yazılım Mühendisliği

Ders Kodu: YZM104 - Programlama II

Öğretim Elemanı: Dr. Öğr. Üyesi İrfan KÖSESOY

Proje Numarası | Adı: Proje 5 | Whack-a-Mole

Hazırlayan: Murat Uymaz – 240229026

1. Projenin Amacı

Bu projenin temel amacı; Kocaeli Üniversitesi Mühendislik Fakültesi Yazılım Mühendisliği Bölümü bünyesinde yürütülen YZM104 - Programlama II dersinin gereksinimleri doğrultusunda , nesne yönelimli programlama prensiplerini somut bir mühendislik çıktısına dönüştürmektir. Proje kapsamında derleme ve grafik kütüphanelerinden yararlanılarak, refleks ve hıza dayalı tek oyunculu bir arcade klasiği olan “Köstebek Vurmaca” oyunu geliştirilmiştir.

Oyunun temel işleyişi, ekran üzerinde belirli bir geometrik düzende konumlandırılmış 9 adet delikten, rastgele zaman aralıklarıyla ve dinamik hızlarda beliren köstebek figürlerine dayanmaktadır. **Oyuncunun temel görevi**, bu köstebekler deliklere geri girmeden önce fare aracılığıyla üzerlerine tıklama yaparak kısıtlı süre içinde en yüksek skoru elde etmektir. 60 saniyelik bir geri sayım mekanizmasının işletildiği bu süreçte , zamana karşı yarışarak kullanıcının refleksleri test edilmektedir.

1. **Grafiksel Kullanıcı Arayüzü :** Kullanıcı etkileşimini artıracak görsel bir kütüphane yardımıyla pencereli, doku ve yazı fontu içeren modern bir arayüz mimarisi inşa edildi.
2. **Gerçek Zamanlı Olay Yönetimi :** Kullanıcıdan gelen fare tıklamalarını ve klavye komutları bir oyun döngüsü içerisinde anlık olarak yakalandı ve işlendi.
3. **Çarpışma Algılama :** Ekranda tıklanan piksellerin koordinatları ile hareketli köstebeklerin anlık matematiksel sınırlarını belirleyerek başarılı vuruşları hassas bir şekilde tespit edildi.
4. **Durum Yönetimi ve Veri Kalıcılığı:** Oyunun ana menü, oynanış, yüksek skorlar ve oyun sonu ekranları arasındaki geçişlerini kararlı bir **Durum Makinesi** tasarımıyla yöneterek elde edilen skorların yerel dosya sisteminde kalıcı olarak depolanması sağlandı ve en yüksek skor sıralaması yapıldı.

2. Kullanılan Teknolojiler ve Araçlar

1. **Programlama Dili:** Projenin temel donanım etkileşimleri ve nesne yönelimli yapısı için standart C++ kullanılmıştır.
2. **Grafik ve Ses Kütüphanesi:** Çapraz platform destekli *SDL2* kütüphanesi tercih edilmiştir. Görüntü dosyalarının (.png, .jpg) işlenmesi için *SDL2_image*, metin ve font işlemleri için *SDL2_ttf*, arkaplan müziği ve çarpışma ses efektleri için ise *SDL2_mixer* alt modülleri projeye eklenmiştir.
3. **Derleme ve İnşa Sistemi:** Projenin farklı işletim sistemlerinde sorunsuz derlenebilmesi için CMake sistemi kullanılmış; kaynak dosyaları ve kütüphane bağlantıları bu sistem üzerinden yapılandırılmıştır.

3. Yazılım Mimarisi ve Teknik Detaylar

Proje, kod tekrarını önlemek ve sürdürülebilir bir yapı kurmak adına çeşitli mühendislik yaklaşımlarıyla tasarlanmıştır.

1. **Durum Makinesi Mimarisi:** Oyunun genel akışı; Menü (MENU), Oynanış (PLAYING), Oyun Sonu (GAME_OVER) ve Yüksek Skorlar (HIGH_SCORE) olmak üzere 4 temel duruma ayrılmıştır. update ve render fonksiyonları, mevcut duruma göre dinamik olarak çalışarak ekranlar arası geçişleri ve bellek yönetimini kolaylaştırmıştır.
2. **Katmanlı Çizim Kuyruğu (Render Queue):** Ekranı çizilecek dokuların (köstebek, delik, menü butonları, arkaplan) doğru derinlik sırasıyla çizilmesi için özelleştirilmiş bir renderQueue veri yapısı geliştirilmiştir. Çizilecek her objeye bir Z-indeksi atanmış ve çizim işlemi öncesi bu indekse göre otomatik sıralama yapılarak görsel hataların (örneğin köstebeğin deliğin önünde durması gerekirken arkasında kalması) önüne geçilmiştir.
3. **Zaman Yönetimi:** Oyun döngüsü, donanımların FPS değerinden bağımsız hale getirilmiştir. Önceki kare ile mevcut kare arasındaki zaman farkı hesaplanarak animasyonların, köstebek çıkış hızlarının ve 60 saniyelik oyun süresinin her bilgisayarda eşzamanlı ve akıcı bir şekilde çalışması sağlanmıştır.
4. **Veri Kalıcılığı:** Oyuncuların elde ettiği skorlar, standart dosya okuma/yazma kütüphaneleri yardımıyla *highscores.txt* dosyasına yazdırılmış ve okunmuştur. Bu sayede oyun kapatılıp açılrsa dahi skor tablosunun kalıcılığı garanti altına alınmıştır.

4. Sınıflar

1. Game Sınıfı:

4.1.1. Temel Döngü ve Kurulum

- **game():** Sınıfın kurucu metodudur. Nesne oluşturulduğunda ilk çağrılan fonksiyondur, temel başlangıç atamalarını yapar.
- **~game():** Sınıfın yıkıcı metodudur. Oyun nesnesi bellekten silinirken otomatik çağrılır ve içeride shutdown() fonksiyonunu tetikleyerek bellek sızıntılarını önler.
- **init():** Oyunun temelidir. SDL2 kütüphanesini, pencereyi, çiziciyi, ses ve yazı tipi eklentilerini başlatır. Gerekli tüm görselleri, sesleri ve butonların koordinatlarını ayarlar. Başarılı olursa true döndürerek oyunun çalışmaya hazır olduğunu bildirir.
- **run():** Oyunun ana döngüsüdür. Oyun açık kaldığı sürece saniyede defalarca kez çalışır. Her dönüşünde geçen süreyi hesaplar ve sırasıyla processInput(), update() ve render() fonksiyonlarını çağırır.
- **shutdown():** Oyundan çıkıldığında RAM'de yer kaplayan tüm dokuları, ses dosyalarını, yazı tiplerini ve SDL penceresini bellekten güvenli bir şekilde silerek sistemi temizler.

4.1.2. Oyun Mantığı

- **processInput():** Klavyeden ve fareden gelen anlık hareketleri yakalar. F11 ile tam ekran yapma, penceredeki çarpı tuşuna tıklayarak oyunu kapatma, butonlara tıklama veya oyun esnasında (PLAYING durumundayken) fare sol tiki ile köstebekleri vurma işlemlerini takip eder ve ilgili tepkileri verir.
- **update():** Oyunun kalbidir. Zamanlayıcıları kontrol eder. 60 saniyelik oyun süresini geriye sayar, rastgele aralıklarla köstebeklerin çıkış sürelerini belirler, köstebeklerin ekranda kalma sürelerini hesaplar ve süre bittiğinde durumu GAME_OVER olarak günceller.
- **render():** Ekrandaki her şeyi temizler, oyunun o anki durumuna (Menü, Oyun İçi vs.) bakarak ilgili yardımcı çizim fonksiyonunu çağırır ve son olarak RQ.flush() ile çizimleri ekrana yansıtır.

4.1.3. Çizim Yardımcı Fonksiyonları

- **renderMenu():** Oyun ana menüdeyken arkaplanı, oyunu başlatma, skor tablosu, müzik ve çıkış butonlarını çizim kuyruğına ekler.
- **renderPlaying():** Oyun oynanırken arkaplanı, delikleri, aktif köstebekleri (normal veya vurulmuş haliyle), kalan süreyi, mevcut skoru ve oyun içi butonları kuyruğına ekler.

- **renderGameOver():** Oyun bittiğinde arka planda oyun alanını bulanıklaştırarak üzerine "GAME OVER" tablosunu, final skorunu ve "Tekrar Oyna / Menü" butonlarını çizer.
- **renderHighScore():** "High Scores" butonuna tıklandığında skor tablosu arkaplanını ve dosyadan okunan en yüksek 5 skoru ekrana metin olarak yazdırır.

4.1.4. Veri ve Durum Yönetimi

- **resetRoundState():** Yeni bir oyun başlatıldığında veya oyun tekrarlandığında skoru sıfırlar, süreyi tekrar 60 saniyeye ayarlar ve tüm köstebekleri deliklerine geri sokarak görünmez yapar.
- **checkNewHighScore():** Oyun bittiğinde elde edilen final skorunu alır ve silinmemesi için highscores.txt dosyasının sonuna ekler. Ardından listeyi güncellemek için loadHighScores() fonksiyonunu çağırır.
- **loadHighScores():** Skor dosyasını açar, içindeki tüm skorları okuyarak bir listeye alır. Bu listeyi büyükten küçüğe doğru sıralar ve ekranda göstermek üzere sadece en yüksek ilk 5 skoru hafızada tutar.
- **LoadTexture():** Dışarıdan dosya yolu verilen bir görseli (.png, .jpg vb.) önce SDL_Surface olarak belleğe alır, ardından ekran kartının işleyebileceği SDL_Texture formatına çevirir ve geri döndürür. Görsel yükleme işlemlerindeki kod tekrarını azaltır.

2. Buton Yapısı:

- **isClicked():** Kullanıcının fare ile tıkladığı X ve Y koordinatlarını parametre olarak alır. Bu koordinatların, butonun ekrandaki sınırlarını belirleyen dikdörtgen (rect) alanının içinde olup olmadığını matematiksel olarak kontrol eder. İndeysse true, dışındaysa false döndürerek butona tıklanıp tıklanmadığını tespit eder.

3. renderQueue Fonksiyonları:

- **add():** Ekrana çizilecek olan bir dokuyu Z-indeksi, kaynak ve hedef dikdörtgenleriyle birlikte çizim kuyruğına ekler.
- **clear():** Kuyruktaki tüm elemanları temizler.
- **flush():** Kuyrukta biriken tüm çizimleri ekrana basar. Basmadan önce elemanları Z-indeksine göre (küçükten büyüğe) sıralar, böylece arkada kalması gereken nesneler önce, önde durması gereken nesneler sonra çizilir. Çizim işleminden sonra kuyruğı temizler.

5. Gereksinimlerin Karşılanması ve Uygulama

1. **Rastgelelik Sistemi:** Köstebeklerin ortaya çıkış süreleri ve çıkacakları 3x3'lük delik konumları, rand() fonksiyonu kullanılarak her defasında benzersiz olacak şekilde randomize edilmiştir. Ayrıca ekranda kalma süreleri dinamik bir zamanlayıcı ile kontrol edilmiştir.
2. **Kullanıcı Etkileşimi ve Çarpışma Testi :** Klavye ve fare olayları SDL_PollEvent ile takip edip, fare sol tıklandığında farenin anlık X ve Y koordinatları alınarak, o an ekranda aktif olan köstebeğin SDL_Rect sınırları içinde olup olmadığı matematiksel olarak doğrulanmıştır. Doğru vuruşlarda skor anında güncellenmiş, köstebek "vurulma" görseline geçiş yapılmış ve vurulma ses efekti oynatılmıştır.
3. **Modüler Arayüz:** Oyun içi arayüzde tıklanabilir butonlar için özel bir Button yapısı oluşturulmuş; fare koordinatları ile buton etkileşimleri bu yapı üzerinden modüler olarak kontrol edilmiştir.

6. Proje Teslim Bağlantıları

- **GitHub Repository:** <https://github.com/muratuymaz/whack-a-mole-game>
- **Demo Videosu:** <https://youtu.be/zmi6awxr1ZM>

7. Kaynakça

1. SDL2/Tutorials - SDL Wiki, <https://wiki.libsdl.org/SDL2/>
2. State Machines - Lazy Foo' Productions, <https://lazyfoo.net/articles/article06/index.php>
3. BTK Akademi 2D Oyun Motoru Geliştirme Eğitimi, <https://www.btkakademi.gov.tr/portal/course/2d-oyun-motoru-gelistirme-47816>
4. Microsoft Copilot VS Code Yapay Zeka Kod Asistanı
5. Google Gemini Yapay Zeka Asistanı, <https://gemini.google.com/app>
6. OpenAI ChatGPT Yapay Zeka Asistanı, <https://chatgpt.com>

8. Sertifika

BTK Akademi üzerinden ücretsiz ve Türkçe olarak sunulan “2D Oyun Motoru Geliştirme” eğitimi içeriğinde SDL kütüphanesi kullanımı ile projeyi geliştirme sürecimde ana eğitim kaynağım oldu ve eğitimi tamamlayarak katılım sertifikası kazanmaya hak kazandım.

